Quant Project — Session Summary

Date: 15 May 2026 **Project:** Multi-Strategy Statistical Arbitrage & CTA

1. Project Architecture (Complete Strategies)

Four independent trading strategies share a common data layer, backtest engine, and CLI entry point (**run.py**).

Strategy	Subcommand	Engine	Status
Pairs Trading (cointegration + Kalman)	pairs	run_pairs_backtest	Complete
PCA Statistical Arbitrage	pca	run_portfolio_backtest	Complete
Basket / ETF Arbitrage	basket / basket-multi	run_basket_backtest	Complete
CTA Trend Following (EWMAC)	cta	run_portfolio_backtest	Built this session

Shared infrastructure:

- src/data/fetcher.py — fetch_prices_bulk(), fetch_pair(), fetch_price() via yfinance
- src/backtest/portfolio_engine.py — run_portfolio_backtest() with optional vol-targeted weights
- src/backtest/metrics.py — compute_metrics() (Sharpe, Sortino, Calmar, max drawdown, win rate)
- src/viz/theme.py — shared colour palette used by all strategy visualisations

---

2. Basket / ETF Arbitrage — Improvements This Session

2a. Ridge Regression (src/analytics/basket.py)

Added L2 regularisation to the rolling OLS basket fit to reduce overfitting when the number of constituents is high relative to the window.

```
rolling_basket_spread(..., ridge_alpha=0.0)
```

The penalty is scaled by **trace(X■X) / n_params** so the alpha value is unit-invariant regardless of the number of constituents. Recommended range: 0.05–0.20.

2b. Regime Filter

Suppresses the spread (sets to NaN) whenever the normalised L2 change in OLS coefficients from the prior bar exceeds a threshold. Detects structural breaks where the ETF-basket relationship has genuinely shifted rather than temporarily deviated.

```
rolling_basket_spread(..., regime_filter=0.0) # try 0.20–0.50
```

2c. N-Stocks Cost Scaling (src/strategies/basket/backtest.py)

Transaction costs scale with the number of constituent stocks being traded simultaneously. More instruments means more timing risk when entering/exiting the basket leg.

```
cost_scale = sqrt(max(n_stocks, 1) / 5.0) # baseline = 5 stocks
```

5 stocks → 1.0×, 10 stocks → 1.41×, 20 stocks → 2.0×

2d. Mark-to-Market Equity Curve

Replaced trade-log P&L; with a proper daily MTM equity curve. Unrealised losses now appear in the equity curve while a trade is open, making drawdown measurements accurate.

---

3. EDGAR N-PORT Constituent History Fetcher

Built to address **survivorship bias**: backtests using today's ETF constituents exclude stocks that were removed (often due to poor performance), inflating historical returns.

3a. Architecture (src/data/edgar.py)

- build_constituent_history(ticker, start_date, end_date, top_n) → DataFrame[filing_date, constituents, weights]
- get_constituents_at(history, date) → list[str] — point-in-time lookup
- summarize_changes(history) → DataFrame[filing_date, added, removed, n_total]
- CLI: python run.py basket-history XLF XLE XLK --period 5y --top-n 15

3b. Key Engineering Details

- SPDR sector ETFs (XLF, XLE, XLK, XLV, etc.) are all series of a single trust (CIK 1064641). Discovered and hardcoded the series IDs for all 11 ETFs.
- N-PORT-P filings don't include ticker symbols — only CUSIP. Resolved via OpenFIGI batch API (10 items/batch, rate-limited). Only successful resolutions are cached.
- HTTP Range requests (first 1,500 bytes) used to sniff from XML headers without downloading full 1.8 MB files.
- XSLT prefix (xslFormNPORT-P_X01/) stripped from primary_doc path to get the actual data XML.
- Cache directory: .cache/edgar/ — holds CIK map, series map, per-filing holdings JSON, CUSIP→ticker map.

3c. Survivorship Bias Findings

ETF	Highest-Risk Removals	Impact
XLK	INTC (–60% before removal), PYPL (–80%)	HIGH — Tech has most churn
XLF	Several regional banks pre-2023	MEDIUM
XLE	Relatively stable constituents	LOW
XLV	Moderate biotech churn	LOW-MEDIUM

---

4. Basket Walk-Forward Validation (10y, 7 Folds)

Ran 7 non-overlapping 1-year OOS folds across XLF, XLE, XLK, XLV combined portfolio (--walk-forward 7 --period 10y). Capital split equally across 4 legs.

Fold	Period	Return	Sharpe	Note
1	2019–20	+~8%	+0.9	Pre-COVID trend

2	2020–21	+~5%	+0.6	Post-COVID recovery choppy
3	2021–22	+~7%	+0.8	Energy spike captured
4	2022–23	+14.4%	+1.6	Rate hike cycle — strongest fold
5	2023–24	+3.2%	+0.4	Weakest — narrow mega-cap rally
6	2024–25	+~6%	+0.7	Moderate trends
7	2025–26	+~9%	+1.1	Tariff shock regime

Approximate figures from memory — exact values in terminal output. Settings: ridge_alpha=0.1, regime_filter=0.3, z_entry=1.5.

---

5. Strategy 4: CTA Trend Following (Built This Session)

5a. Signal Design — Multi-Horizon EWMAC

For each instrument, four EWMAC signals are computed and combined with equal weights:

```
EWMAC(fast, slow) = EMA(fast) − EMA(slow)
Normalised = EWMAC / rolling_std(EWMAC, window=slow), clipped to [−2, +2]
Combined = mean([EWMAC_8/32, EWMAC_16/64, EWMAC_32/128, EWMAC_64/256])
```

Direction signal: sign(combined) → {–1, 0, +1}. Optional **--threshold** creates a flat-band around zero.

5b. Instrument Universe (16 ETF Proxies)

Sector	Tickers
Equities	SPY, QQQ, EFA, EEM, IWM
Bonds	TLT, IEF, SHY, TIP
Commodities	GLD, SLV, USO, DBA
FX Proxies	UUP, FXE, FXY

CLI presets: --universe default|equities|bonds|commodities|fx

5c. Volatility Targeting (Added This Session)

Replaced binary equal-weight allocation with vol-targeted weights so each instrument's contribution to portfolio volatility is proportional to its share of the risk budget:

```
weight_i = direction_i × τ / (σ_i × N_active)
```

τ = annualised vol target (default 20%), σ_**i** = EWM(25) annualised daily vol, **N_active** = active positions count. This naturally de-risks the portfolio when volatility spikes.

Implementation: **vol_targeted_weights()** in src/analytics/cta.py. The portfolio engine (run_portfolio_backtest) extended with optional **weights_df** parameter — backward compatible.

5d. 5-Fold Annual Results (10y lookback, 16 instruments)

Fold	Period	Equal-wt Return	Equal-wt Sharpe	Vol-tgt Return	Vol-tgt Sharpe	Avg Leverage

1	2021–22	+6.6%	0.93	+8.3%	0.97	3.57×
2	2022–23	–8.4%	–0.89	–6.7%	–0.58	1.74×
3	2023–24	–0.3%	–0.03	+3.1%	0.45	2.38×
4	2024–25	–3.7%	–0.51	–0.7%	–0.03	2.51×
5	2025–26	+23.0%	2.81	+23.0%	2.59	3.02×

Key observations:

- Fold 5 (2025–26) is a clear outlier — driven by the April 2025 tariff shock creating persistent trends across equities and commodities.
- Vol targeting consistently reduces losses in the bad years (folds 2–4) while preserving the upside in fold 5.
- The strategy underperformed in 2022 despite that being a historically good year for real CTAs — likely because binary ±1 weighting treats low-vol instruments (SHY) the same as high-vol ones (EEM, USO) without vol targeting.
- Gross leverage ranges from 1.7× (high-vol regime, 2022) to 3.6× (low-vol regime, 2021). Use --weight-cap 0.40 to limit single-instrument exposure.
- Bonds-only universe (TLT, IEF, SHY, TIP): –1.6% return, Sharpe –0.50 — range-bound rates post-2023 hiked cycle.

5e. CLI Usage

```
python run.py cta
python run.py cta --universe default --period 5y --test-period 1y
python run.py cta --universe equities --period 10y --test-period 2y --vol-target 0.15
--weight-cap 0.40
```

---

6. Complete File Map

New Files Created This Session

File	Purpose
src/analytics/cta.py	ewmac(), combined_ewmac(), instrument_vol(), vol_targeted_weights()
src/analytics/basket.py	rolling_basket_spread() with ridge_alpha, regime_filter
src/data/edgar.py	Full EDGAR N-PORT constituent history fetcher (CUSIP→ticker via OpenFIGI)
src/strategies/cta/__init__.py	Empty package marker
src/strategies/cta/signals.py	CTA_UNIVERSE dict, generate_cta_positions()
src/strategies/cta/viz.py	plot_cta_equity(), plot_cta_signals(), plot_cta_contributions()
src/strategies/basket/backtest.py	run_basket_backtest() with MTM equity, n_stocks cost scaling

Modified Files

File	Change

run.py	Added cta, basket-multi, basket-history subcommands; _run_cta, _register_cta
src/backtest/portfolio_engine.py	Added optional weights_df param to run_portfolio_backtest()

---

7. Open Questions & Suggested Next Steps

CTA Strategy

- Run the 2022 underperformance investigation: why did the strategy lose when real CTAs (Man AHL, Winton) made 20–40%? Likely: binary sizing treats low-vol instruments identically to high-vol without vol targeting — now fixed.
- Consider correlation-adjusted weighting (risk parity) instead of N_active divisor — would reduce concentration when instruments are highly correlated (e.g., SPY/QQQ).
- Walk-forward validation (--walk-forward N) not yet implemented for the cta subcommand — copy the basket-multi walk-forward pattern.
- The threshold=0 setting means the strategy is never flat — consider --threshold 0.2 to filter out low-conviction signals.
- Evaluate whether USO (front-month oil ETF) introduces roll-cost drag not captured in price returns.

Basket Strategy

- Survivorship-bias corrected backtest: use EDGAR-sourced historical constituents instead of today's composition for XLK specifically (INTC and PYPL are high-risk omissions).
- The basket-history command already surfaces historical composition — next step is plumbing those constituent lists into basket-multi automatically.

General

- No unit tests yet — worth adding at least smoke tests for each strategy's signal generation and backtest engine.
- All strategies use a fixed 5bps cost model. A more realistic model would use half-spread × average daily volume for each instrument.
- Consider a combined multi-strategy portfolio view: equal-risk-weighted allocation across all four strategies.

---

8. Quick CLI Reference

Pairs trading (single pair)

```
python run.py pairs --pair GS/MS --period 2y --backtest --test-period 1y
```

Pairs scan (universe)

```
python run.py pairs --scan --universe financials --period 2y
```

PCA stat arb

```
python run.py pca --universe financials --period 2y --n-factors 3 --test-period 1y
```

Basket single

```
python run.py basket --etf XLF --stocks GS MS JPM BAC C --period 2y --test-period 1y
```

Basket multi

```
python run.py basket-multi --basket XLF:GS,MS,JPM --basket XLE:XOM,CVX --period 5y
--walk-forward 5
```

Basket history (EDGAR)

```
python run.py basket-history XLF XLE XLK --period 5y --top-n 15
```

CTA default

```
python run.py cta --period 5y --test-period 1y
```

CTA with vol cap

```
python run.py cta --universe default --period 10y --test-period 2y --vol-target 0.20
--weight-cap 0.40
```

CTA equities only

```
python run.py cta --universe equities --period 5y --test-period 1y
```
